

Relatório de Auditoria de Segurança

Revisão estática orientada a cinco classes de risco

DATA

1º de setembro de 2026

ESCOPO

API Fastify/Prisma, frontend Next.js/React, PostgreSQL, Docker Compose, GitHub Actions e histórico Git completo.

MÉTODO

Isolamento e posse, autorização servidor/UI, IDOR, segredos hardcoded/históricos e sinks de XSS.

4 achados verificados • 2 altos • 2 médios

Nenhum achado crítico. Nenhum IDOR ou XSS confirmado.

Stack detectada e metodologia

Backend	TypeScript 6, Fastify 5, Zod 4, Prisma 7 e PostgreSQL. Consultas ORM e uma query SQL parametrizada com tagged template do Prisma.
Autenticação e autorização	JWT assinado por @fastify/jwt. Access token de 15 minutos; refresh token em cookie HttpOnly/SameSite=Strict com rotação persistida. RBAC ADMIN/MEMBER em middleware de rota.
Frontend	Next.js 16, React 19 e TypeScript. Access token mantido em memória. JSX fornece escape padrão; CSP com nonce e cabeçalhos defensivos.
Deploy e automação	Vercel (API Fastify), Docker Compose para PostgreSQL local e GitHub Actions para testes unitários/E2E. Não há Helm ou Terraform.

Mapeamento das cinco categorias

- 1. Isolamento: o domínio não possui tenant/workspace. Academias são globais; dados pessoais de check-in usam filtro manual por user_id derivado do JWT.
- 2. Permissões: cada gate role/isAdmin do React foi cruzado com a rota Fastify e sua cadeia controller → service → repository.
- 3. IDOR: todos os 26 handlers HTTP foram inventariados; parâmetros gymId/checkInId e IDs vindos do JWT foram revisados.
- 4. Segredos: árvore atual, dotfiles rastreados, deploy/CI, bundle .next e todas as revisões Git foram pesquisados.
- 5. XSS: sinks HTML/JS/URL, markdown/templates, escape do React, validação de telefone e CSP foram verificados.

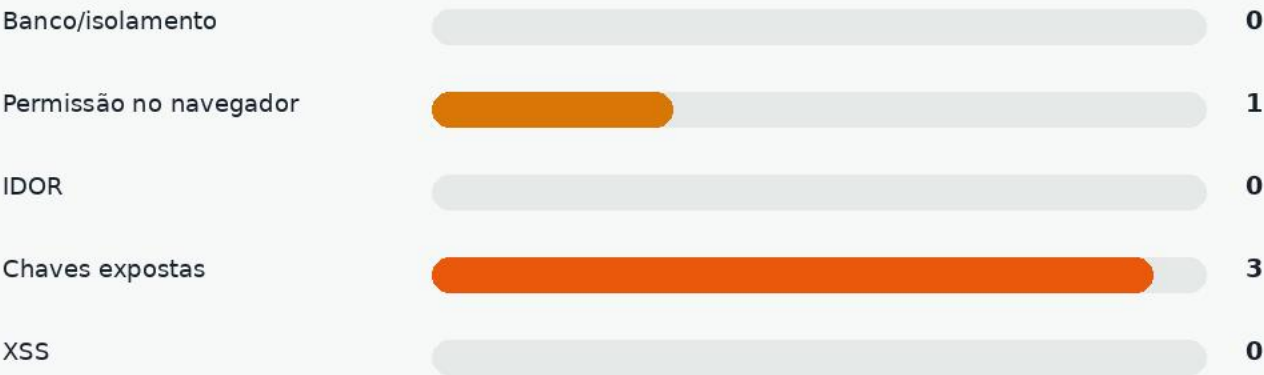
Resumo executivo

Foram confirmados quatro achados: dois de severidade alta e dois de severidade média. O risco dominante é a possibilidade de implantação com segredo JWT público ou fraco; nesse cenário, um atacante pode assinar um token com role ADMIN. A autorização administrativa atual está corretamente no servidor, com uma exceção: a UI restringe check-in a MEMBER, mas a API aceita ADMIN.

Distribuição por severidade



Achados por categoria



Conclusão

A postura geral de autorização e XSS é boa. A prioridade imediata é impedir segredos JWT conhecidos/fracos e confirmar a rotação do valor histórico. Em seguida, alinhar a regra MEMBER no endpoint de check-in e endurecer o Compose local.

Pontos fortes e pontos fracos

Pontos fortes — com evidência

- Dados pessoais isolados pelo sujeito autenticado: history usa req.user.sub (controller linhas 13-16) e o repositório filtra user_id (linhas 44-57).
- Métrica pessoal usa req.user.sub (get-user-check-ins-count.ts:8-10) e count com where user_id (prisma-check-ins-repository.ts:136-142).
- Perfil ignora qualquer ID externo e carrega apenas req.user.sub; password_hash é removido da resposta (profile.ts:7-14).
- Todas as 13 operações administrativas cruzadas usam verifyJWT + verifyUserRole("ADMIN") nas rotas de academias e check-ins.
- IDs de rota são validados como UUID por Zod; SQL geoespacial usa interpolação parametrizada do Prisma, não concatenação.
- Refresh token é HttpOnly, SameSite=Strict, rotacionado atomicamente e vinculado ao user_id; access token fica em memória no frontend.
- Não há dangerouslySetInnerHTML, innerHTML, eval, new Function, markdown ou template de e-mail. React escapa os valores persistidos.
- O href tel: recebe telefone restrito por regex no backend (gym-schema.ts:12-18), impedindo esquema javascript:.
- CSP restringe scripts por nonce e não permite object-src; unsafe-eval existe apenas em desenvolvimento (proxy.ts:6-17).

Pontos fracos centrais

- Segredo de assinatura publicamente conhecido no histórico e nenhuma política de entropia/denylist no startup.
- Credenciais docker/docker com PostgreSQL publicado em todas as interfaces pelo Compose.
- A restrição MEMBER para check-in existe na renderização, mas não na rota da API.

Tabela de achados

Somente achados demonstráveis no código foram incluídos. Condições de explorabilidade são explicitadas para evitar tratar configuração local como comprometimento de produção.

ALTA	F-01 Chave JWT pública permanece no histórico Git <code>.env.example@0427630:2</code> Um valor de assinatura com aparência de segredo foi versionado em um arquivo de exemplo. Se ele foi reutilizado em qualquer ambiente, qualquer pessoa com acesso ao repositório pode forjar JWTs, inclusive com role ADMIN.
ALTA	F-02 Startup aceita JWT_SECRET fraco ou conhecido <code>apps/api/src/env/index.ts:7</code> A validação exige apenas string; aceita vazio lógico curto, 'testing' ou o placeholder público do exemplo. Uma implantação que copie esse valor permite falsificação de tokens.
MÉDIA	F-03 Restrição MEMBER de check-in existe só no frontend <code>apps/web/.../discovery-island.tsx:192; apps/api/.../check-ins-routes.ts:15</code> O botão é exibido apenas para MEMBER, mas o servidor exige somente JWT. Um ADMIN autenticado pode chamar POST /gyms/:gymId/check-ins diretamente.
MÉDIA	F-04 PostgreSQL local publicado com credenciais previsíveis <code>apps/api/docker-compose.yml:4-8</code> A porta 5432 é publicada no host e as credenciais são docker/docker. Em host compartilhado ou acessível por rede, outro usuário pode autenticar e ler/alterar o banco local.

F-01 — Chave JWT pública no histórico

ALTA

Evidência

.env.example na revisão 0427630f708bef00aa86df99fb98126bb50c8969, linha 2

```
2 JWT_SECRET="[REDACTED – valor comprometido no histórico]"
```

Por que é explorável

O valor completo está disponível a qualquer leitor do histórico. Como @fastify/jwt usa JWT_SECRET para verificar access e refresh tokens, reutilizar esse valor permite assinar {type: 'access', role: 'ADMIN', sub: <uuid>} e alcançar rotas administrativas.

Condição de explorabilidade

Escopo

Explorável se esse exemplo histórico foi usado em Vercel, homologação ou produção. A auditoria não prova que o valor local/produção atual seja o mesmo; por isso é necessário confirmar e, na dúvida, rotacionar.

F-02 — Política insuficiente para JWT_SECRET

ALTA

Evidência

apps/api/src/env/index.ts, linhas 5-9; apps/api/.env.example, linha 2

```
5 const envSchema = z.object({
6   NODE_ENV: z.enum([...]).default("dev"),
7   JWT_SECRET: z.string(),
8   PORT: z.coerce.number().default(3333),
9   DATABASE_URL: z.string(),
10
11   // JWT_SECRET=replace-with-a-long-random-secret
```

Por que é explorável

O processo inicia com qualquer string. O valor público do .env.example também satisfaz o schema. Quem conhecer um segredo efetivamente implantado consegue forjar o papel ADMIN, pois o papel é lido do payload assinado nas linhas 5-8 de verify-user-role.ts.

Condição de explorabilidade

Escopo

Explorável quando a configuração usa valor curto, previsível ou copiado do exemplo. Corrigir com min(32/64), rejeição explícita de placeholders/test values em production e segredo gerado criptograficamente.

F-03 — Gate MEMBER somente na interface

MÉDIA

Evidência

apps/web/src/features/discovery/discovery-island.tsx, linhas 190–193; apps/api/src/http/routes/check-ins-routes.ts, linha 15

```
190 <GymGrid
191   gyms={gyms}
192   allowCheckIn={user.role === "MEMBER"}
193   checkIn={checkIn}

15 app.post("/gyms/:gymId/check-ins", { onRequest: [verifyJWT] }, checkIn);
```

Por que é explorável

A UI oculta a ação para ADMIN, mas um administrador pode enviar a mesma requisição HTTP diretamente. O controller usa req.user.sub e o serviço cria o registro sem validar role. Isso viola a separação de papéis expressa na interface e cria check-ins administrativos fora do fluxo esperado.

Condição de explorabilidade

Escopo

Explorável por qualquer conta ADMIN com token válido, desde que envie coordenadas a até 100 m e ainda não tenha feito check-in no dia. Adicionar `verifyUserRole("MEMBER")` à rota e teste E2E de negação para ADMIN.

F-04 — Banco local com credencial previsível

MÉDIA

Evidência

apps/api/docker-compose.yml, linhas 3-8

```
3 image: bitnami/postgresql
4 ports:
5   - "5432:5432"
6 environment:
7   POSTGRESQL_USERNAME: docker
8   POSTGRESQL_PASSWORD: docker
```

Por que é explorável

O bind sem endereço publica 5432 nas interfaces do host. Com usuário e senha conhecidos no repositório, um processo local não confiável — ou um host alcançável pela rede, conforme firewall — pode autenticar e manipular dados do ambiente.

Condição de explorabilidade

Escopo

Condicionado à execução do Compose em máquina compartilhada ou com 5432 acessível. Para desenvolvimento individual isolado o impacto é menor. Usar 127.0.0.1:5432:5432, credencial via .env não versionado e volume/rede dedicados.

Cobertura e resultados negativos

Inventário de rotas — 26 handlers

Públicas / sessão (6) GET /, GET /health, POST /users, POST /sessions, POST /sessions/refresh, POST /sessions/logout
Usuário autenticado (7) GET /me, GET /gyms, GET /gyms/search, GET /gyms/nearby, POST /gyms/:gymId/check-ins, GET /check-ins/history, GET /check-ins/metrics
Administrador (13) POST /gyms; GET /gyms/deleted; PATCH/DELETE /gyms/:gymId; DELETE /gyms/deleted/permanent; DELETE /gyms/:gymId/permanent; PATCH /gyms/:gymId/restore; GET /check-ins/pending expired validated; GET /check-ins/metrics/global; DELETE /check-ins/expired/:checkInId; PATCH /check-ins/:checkInId/validate

Contagem total: 6 públicas/sessão + 7 autenticadas + 13 administrativas = 26. Rotas com vários verbos foram contadas por handler registrado.

Categorias sem achado confirmado

Banco sem tranca / isolamento Não há tenant no modelo. Academias são globais por desenho. Histórico, métrica e perfil pessoais recebem exclusivamente req.user.sub; os repositórios aplicam user_id. Listagens com PII são administrativas.
IDOR Todos os IDs externos foram revisados. Recursos pessoais não aceitam userId do cliente. gymId/checkInId designam recursos globais e suas mutações estão protegidas por ADMIN; check-in novo vincula o usuário do JWT.
XSS Nenhum sink de HTML/JS foi encontrado. Conteúdo persistido é interpolado por JSX com escape. O único URL dinâmico é tel: e o backend restringe o telefone a dígitos/formatação conhecida.

Recomendações priorizadas

P1**Rotacionar e invalidar segredos JWT**

Confirmar se o valor histórico foi usado em qualquer ambiente. Em caso positivo ou dúvida, trocar `JWT_SECRET` e revogar sessões existentes imediatamente.

P1**Aplicar política de segredo no startup**

Exigir pelo menos 32 bytes aleatórios (preferível 64), negar placeholders e valores de teste quando `NODE_ENV=production`, e documentar geração segura.

P2**Autorizar MEMBER no endpoint de check-in**

Adicionar `verifyUserRole("MEMBER")` e teste E2E garantindo 403 para ADMIN. Manter a UI como conveniência, não como controle.

P2**Restringir PostgreSQL do Compose**

Bind em `127.0.0.1`, senha local fora do Git e documentação explícita de que o Compose não deve ser usado como configuração de produção.

P3**Adicionar testes negativos de matriz de papéis**

Para cada rota administrativa, testar `MEMBER→403`; para cada rota exclusiva de membro, testar `ADMIN→403`. Automatizar a matriz para evitar regressões.

P3**Automatizar secret scanning**

Executar detector de segredos no pre-commit/CI e bloquear novos valores de alta entropia. Revisar também bundles de produção antes do deploy.

ISSUES PARA O GITHUB — Issue 1

Texto completo em Markdown, pronto para copiar e colar.

--- ISSUE 1 ---

Título: [Segurança] Impedir uso de segredos JWT públicos ou fracos

Labels sugeridas: security, severity:high

Descrição do problema e explorabilidade

O histórico contém uma chave JWT pública em `.env.example@0427630:2` e o schema atual aceita qualquer string em `apps/api/src/env/index.ts:7`. Se um ambiente reutilizar o valor histórico, o placeholder atual ou outro valor previsível, um atacante pode assinar access tokens com role ADMIN e acessar todas as rotas privilegiadas.

Evidência

`.env.example@0427630:2`

`JWT_SECRET="dfheuf...BHFVCE"`

`apps/api/src/env/index.ts:7`

`JWT_SECRET: z.string(),`

Impacto

Falsificação de identidade e elevação completa para ADMIN; leitura de PII de check-ins e alteração/exclusão de academias.

Sugestão de correção

Rotacionar o segredo se houver qualquer chance de reutilização; exigir segredo aleatório com comprimento/entropia adequados; rejeitar valores conhecidos em produção; invalidar sessões antigas após rotação.

Critérios de aceite

- [] JWT_SECRET de produção foi rotacionado e sessões antigas invalidadas.
- [] Startup de produção falha com placeholder, "testing" e segredo curto.
- [] Startup aceita segredo aleatório de pelo menos 32 bytes.
- [] .env.example não contém valor utilizável como assinatura.
- [] Há testes automatizados para a validação de ambiente.

--- FIM ISSUE 1 ---

ISSUES PARA O GITHUB — Issue 2

Texto completo em Markdown, pronto para copiar e colar.

--- ISSUE 2 ---

Título: [Segurança] Validar papel MEMBER no endpoint de check-in

Labels sugeridas: security, severity:medium

Descrição do problema e explorabilidade

O frontend mostra a ação apenas quando `user.role === "MEMBER"` (`discovery-island.tsx:192`), mas `POST /gyms/:gymId/check-ins` exige somente `verifyJWT` (`check-ins-routes.ts:15`). Um ADMIN pode chamar a API diretamente e criar check-in para seu próprio sub, contornando a regra de papel da interface.

Evidência

`apps/web/src/features/discovery/discovery-island.tsx:192`

`allowCheckIn={user.role === "MEMBER"}`

`apps/api/src/http/routes/check-ins-routes.ts:15`

`app.post("/gyms/:gymId/check-ins", { onRequest: [verifyJWT] }, checkIn);`

Impacto

Violação da separação de papéis e criação de registros administrativos fora do fluxo previsto.

Sugestão de correção

Adicionar `verifyUserRole("MEMBER")` depois de `verifyJWT` e cobrir a matriz de papéis em teste E2E.

Critérios de aceite

- [] MEMBER autenticado continua recebendo 201 quando cumpre as regras de negócio.
- [] ADMIN autenticado recebe 403 no POST de check-in.
- [] Usuário sem JWT recebe 401.
- [] Existe teste E2E de regressão para ADMIN→403.

--- FIM ISSUE 2 ---

ISSUES PARA O GITHUB — Issue 3

Texto completo em Markdown, pronto para copiar e colar.

--- ISSUE 3 ---

Título: [Segurança] Restringir PostgreSQL local e remover credencial previsível

Labels sugeridas: security, severity:medium

Descrição do problema e explorabilidade

apps/api/docker-compose.yml publica 5432:5432 e define usuário/senha docker/docker. Em máquina compartilhada ou com a porta alcançável pela rede, qualquer pessoa que conheça o repositório pode autenticar no banco local.

Evidência

apps/api/docker-compose.yml:4-8

ports: ["5432:5432"]

POSTGRESQL_USERNAME: docker

POSTGRESQL_PASSWORD: docker

Impacto

Leitura, alteração ou exclusão de dados do ambiente local; possível apoio a ataques contra testes e desenvolvimento.

Sugestão de correção

Publicar apenas em 127.0.0.1, carregar senha de arquivo .env não versionado e documentar que o Compose é exclusivamente local.

Critérios de aceite

- [] A porta do PostgreSQL faz bind somente em 127.0.0.1 (ou não é publicada).
- [] A senha não está hardcoded no Compose.
- [] O projeto falha com mensagem clara quando a variável local obrigatória está ausente.
- [] A documentação proíbe reutilizar essa configuração em produção.

--- FIM ISSUE 3 ---